

Exercise 2

```
vihan@vihanzVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5
[*] / gcc -fopenmp -O2 -o exe2_single exe2_single.c && OMP_NUM_THREADS=1 ./exe2_single
thread # 0
Inside Single # 0
Exiting Single
Hi again from thread # 0
```

```
vihan@vihanzVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5
/ ) gcc -fopenmp -O2 -o exe2_single exe2_single.c && OMP_NUM_THREADS=2 ./exe2_single
thread # 1
Inside Single # 1
Exiting Single
thread # 0
Hi again from thread # 0
Hi again from thread # 1
```

```
vihan@vihanzVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5
/ ) gcc -fopenmp -O2 -o exe2_single exe2_single.c && OMP_NUM_THREADS=4 ./exe2_single
thread # 0
Inside Single # 0
Exiting Single
thread # 3
thread # 2
thread # 1
Hi again from thread # 2
Hi again from thread # 1
Hi again from thread # 0
Hi again from thread # 3
```

```
vihan@vihanzVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5
$ gcc -fopenmp -O2 -o exe2_single exe2_single.c && OMP_NUM_THREADS=8 ./exe2_single
thread # 1
Inside Single # 1
Exiting Single
thread # 2
thread # 4
thread # 3
thread # 5
thread # 6
thread # 7
thread # 0
Hi again from thread # 1
Hi again from thread # 2
Hi again from thread # 4
Hi again from thread # 3
Hi again from thread # 6
Hi again from thread # 0
Hi again from thread # 7
Hi again from thread # 5
```

Q. Briefly describe how the pragma omp single command works in this program.

#pragma omp single marks a block that only one thread in the team executes (whichever gets there first) the rest skip it entirely. In my run, thread 1 happened to grab it. Importantly, single has an implicit barrier at the end: no thread proceeds past the block until the one thread has finished it. That's why "Inside Single"/"Exiting Single" always fully complete before any thread's "Hi again" line prints.